Design for Performance

- The cost of computer systems is dropping significantly, while their performance and capacity are rising just as dramatically.
- To illustrate this rapid advancement, modern laptops now possess the computing power that large IBM mainframes had just 10 to 15 years ago. Furthermore, microprocessors have become so cheap that we now have disposable microprocessors.
- This immense microprocessor power is necessary for modern desktop applications, which include image processing, 3D rendering, speech recognition, videoconferencing, multimedia authoring, voice and video annotation, and simulation modeling.
- In the enterprise sector, massive client/server networks have replaced the huge mainframe computer centers of the past, with businesses relying on increasingly powerful servers for heavy transaction and database processing.
- Similarly, cloud service providers use massive, high-performance server banks to satisfy high-volume and high-transaction-rate applications for a broad spectrum of clients.

Techniques built into contemporary processors

1. Pipelining

- A technique where the execution of an instruction is broken down into multiple stages (such as fetching, decoding, fetching operands, and performing calculations). Pipelining enables a processor to work on multiple instructions simultaneously by overlapping these different phases for different instructions in a conceptual "pipe".
- This works on the exact same principle as a manufacturing assembly line. For instance, while the processor is busy executing one instruction in the final stage, it is simultaneously decoding the next instruction and fetching a third instruction from memory.

2. Branch Prediction

- A mechanism where the processor looks ahead in the instruction code fetched from memory and predicts which branches, or groups of instructions, are likely to be processed next.
- If the processor guesses correctly most of the time about which path a program will take, it can prefetch the correct instructions ahead of time and buffer them. This ensures the processor has work ready and waiting, keeping it busy instead of waiting for the next instruction to be fetched.

3. Superscalar Execution

- The ability to issue more than one instruction in every single processor clock cycle. In effect, this creates multiple parallel pipelines within a single processor.
- Instead of processing a single instruction stream sequentially, the processor can take multiple instructions that do not depend on one another and execute them completely in parallel during the same clock cycle.

4. Data Flow Analysis

- The processor analyzes which instructions are dependent on each other's results or data. It uses this information to create an optimized schedule of instructions.
- The processor schedules instructions to be executed as soon as they are ready, which operates independently of the original order written in the program. If one instruction is stalled waiting for data, the processor bypasses it to execute subsequent independent instructions, preventing unnecessary delays.

5. Speculative Execution

- Using a combination of branch prediction and data flow analysis, the processor speculatively executes instructions ahead of their actual appearance in the program's execution flow.
- The processor guesses which instructions will be needed next and goes ahead and executes them, holding the results in temporary locations. If the program ultimately requires those instructions, the results are already calculated and ready to go, which keeps the execution engines as busy as possible.

Performance Balance

Performance balance refers to the adjustment and tuning of a computer's organization and architecture to **compensate for the mismatch in capabilities among its various components**.

As processor speeds have raced ahead at breakneck speed, other critical components—particularly main memory and I/O devices—have not kept pace. This creates a significant bottleneck, especially at the interface between the processor and main memory. If the memory pathway fails to keep up with the processor's demands, the processor is forced to stall in a "wait state," wasting valuable processing time.

To achieve performance balance, system architects implement several design strategies:

- **Wider Data Paths:** Increasing the number of bits retrieved at one time by making DRAMs and bus data paths "wider" rather than "deeper".
- **Cache Hierarchies:** Reducing the frequency of main memory access by incorporating increasingly complex and efficient cache structures (like L1, L2, and L3 caches) between the processor and main memory.
- **DRAM Buffering:** Changing the DRAM interface to make it more efficient, such as including a cache or other buffering schemes directly on the DRAM chip.
- **Improved Interconnection:** Increasing the interconnecting bandwidth between processors and memory by utilizing higher-speed buses and a hierarchy of buses to better structure data flow.
- **I/O Management:** Using caching, buffering, and elaborate interconnection structures to handle the massive data throughput demands of modern peripheral devices (like graphics displays and network interfaces) so they do not bottleneck the processor.

Ultimately, designing for performance balance is a constantly evolving process, as different technologies (processors, memory, buses, peripherals) advance at different rates and new applications continually change the demands placed on the system.

Improvements in Chip Organization and Architecture

Three primary approaches designers use to increase processor speed and overall performance:

- **Increase the hardware speed of the processor:** This is fundamentally achieved by shrinking the size of the physical logic gates. By packing more gates tightly together on the chip, the physical distance between them is reduced, which decreases the propagation time for signals and allows for an increased clock rate.
- **Increase the size and speed of caches:** Designers interpose larger and faster cache memories between the processor and main memory. By dedicating a significant portion of the processor chip itself to these caches, cache access times drop significantly.
- **Change processor organization and architecture:** This involves altering the internal design to increase the effective speed of instruction execution. Typically, this is achieved by implementing various forms of parallelism (like the pipelining and superscalar execution discussed earlier).

Problems with Clock Speed and Logic Density

Physical obstacles and limitations that arise when simply trying to increase clock speed and pack more transistors onto a chip:

- **Power (Heat Dissipation):** As logic density and clock speeds increase, the power density also increases. Dissipating the immense heat generated by these high-density, high-speed chips has become a severe design issue.
- **RC Delay:** The speed at which electrons can flow between transistors is physically limited by the resistance (R) and capacitance (C) of the metal wires connecting them. As chip components decrease in size, the interconnecting wires become thinner (which increases resistance) and sit closer together (which increases capacitance). This combined increase in the RC product creates a significant delay in signal transmission.
- **Memory Latency:** Memory access speeds (latency) continue to lag behind processor speeds, meaning a faster processor often just ends up waiting longer for memory.

Multicore Systems

The use of multiple processors on the same chip provides the potential to increase performance without increasing the clock rate.

Feature	Multicore (CPU)	Many Integrated Core (MIC)	Graphics Processing Unit (GPU)
Core Architecture	Highly complex, general-purpose processors built for fast single-thread execution.	Simpler, homogeneous general-purpose processors (like standard x86 cores).	Highly specialized, stripped-down cores optimized for vector mathematics.

Feature	Multicore (CPU)	Many Integrated Core (MIC)	Graphics Processing Unit (GPU)
Core Count	Limited (typically 2 to 16+ cores).	Many (typically 50 to 70+ cores).	Thousands of extremely simple cores.
Execution Style	Parallel, but optimized to handle complex, unpredictable branching tasks quickly.	Highly parallel, running many standard software threads simultaneously.	Massively parallel, running the exact same operation on large sets of data simultaneously.
Primary Use Case	Daily computing, laptops, standard servers, operating systems.	Scientific applications, supercomputing, High-Performance Computing (HPC).	Image processing, gaming, 3D rendering, machine learning, repetitive computation.
Examples	Intel Core i5, AMD Ryzen.	Intel Xeon Phi (Knights Landing).	NVIDIA GeForce RTX, AMD Radeon.

Amdahl's Law and Speedup

Ideal Speedup vs. Real-World Speedup: Ideally, if a single processor takes 1 second to finish a task, N processors should theoretically finish it in $1/N$ seconds (e.g., 2 processors take 0.5 seconds), meaning speedup increases linearly with the number of processors. However, in reality, adding more processors does not lead to a linear increase in speed.

Amdahl's Law: This law states that the speedup of a program using multiple processors is limited by the structure of the program itself. Specifically, it depends on two factors:

1. **Serial (Sequential) Part:** The portion of the program that must be executed one instruction after another (e.g., traversing a linked list). This part cannot be parallelized.
2. **Parallel Part:** The portion of the program that contains concurrent operations (e.g., traversing graphs or trees) and can be distributed across multiple processors.

Formula Derivation

1. Let S = the serial part of the program.
2. Let P = the parallel part of the program.
3. Let N = the total number of processors.
4. **Time for 1 Processor (T_1):**

$$T_1 = S + P$$

If we assume the total time required by a single processor is 1 unit of time, then:

$$1 = S + P$$

Therefore, the serial part can be expressed as:

$$S = 1 - P$$

5. **Time for N Processors (T_N):**

When using N processors, the serial part still takes the same amount of time, but the parallel part's time is divided by N.

$$T_N = S + \frac{P}{N}$$

6. Substituting $S = 1 - P$ into the equation:

$$T_N = (1 - P) + \frac{P}{N}$$

7. **Speedup Formula:**

Speedup is the time required by a single processor divided by the time required by N processors.

$$\text{Speedup} = \frac{T_1}{T_N}$$

Substituting the derived values:

$$\text{Speedup} = \frac{1}{(1 - P) + \frac{P}{N}}$$

Examples Solved

In all the following examples, the total number of processors (N) is 10.

Example 1

- **Given:** Parallel part (P) = 60% (0.6), Sequential part (S) = 40% (0.4)
- **Formula:** Speedup $= \frac{1}{(1-0.6)+\frac{0.6}{10}}$
- **Solution:** Speedup $= \frac{1}{0.4+0.06} =$ **2.17**

(Note: Even with 10 processors, the speedup is only slightly more than double).

Example 2

- **Given:** Parallel part (P) = 80% (0.8), Sequential part (S) = 20% (0.2)
- **Formula:** Speedup $= \frac{1}{(1-0.8)+\frac{0.8}{10}}$
- **Solution:** Speedup $= \frac{1}{0.2+0.08} =$ **3.57**

Example 3

- **Given:** Parallel part (P) = 90% (0.9), Sequential part (S) = 10% (0.1)
- **Formula:** Speedup $= \frac{1}{(1-0.9)+\frac{0.9}{10}}$
- **Solution:** Speedup $= \frac{1}{0.1+0.09} =$ **5.26**

(Note: With 90% of the program parallelized, 10 processors still only make it roughly 5 times faster, not 10 times).

Example 4

- **Given:** Parallel part (P) = 99% (0.99), Sequential part (S) = 1% (0.01)
- **Formula:** Speedup $= \frac{1}{(1-0.99)+\frac{0.99}{10}}$
- **Solution:** Speedup $= \frac{1}{0.01+0.099} =$ **9.17**

(Note: Even in a nearly impossible scenario where 99% of the code is parallel, the speedup using 10 processors is capped at 9.17).

Little's Law

Little's Law is a fundamental and simple mathematical relationship used to analyze queuing systems, which are systems where items wait in line to be served. It can be applied to almost any system that operates in a statistical "steady state" and has no leakage (meaning items do not just disappear from the system).

The law relates three key variables using the formula $L = \lambda W$:

- L: The average number of items currently in the system.
- λ **(Lambda)**: The average rate at which new items arrive at the system.
- W: The average amount of time an item spends in the system.

Benchmark

A benchmark is a program, typically written in a high-level language, that is used to evaluate and measure the performance of a computer system. To be effective, a benchmark program must be representative of a particular programming domain (such as systems programming, numerical programming, or commercial applications), be easily measured, and have wide distribution. Often, a collection of these programs is bundled together into a "benchmark suite" to provide a comprehensive test of a computer's capabilities.

The Need for a Mean

Imagine you want to compare the performance of two computers, PC A and PC B, by running the exact same benchmark program three times on each. You record the following execution times:

- **PC A:** 10s, 20s, 30s
- **PC B:** 5s, 30s, 5s

Looking at these raw numbers, it is impossible to definitively say which system is faster just by comparing individual runs. On the first and third runs, PC B is much faster, but on the second run, PC A is significantly faster. Execution times can vary wildly from one run to another due to variations that are not intrinsic to the program itself, such as disk access time variations or operating system background tasks.

Because a single run or a scattered set of numbers provides only a simplified or ambiguous view of a system's capability, we must use an aggregate single number—a mean (average)—to accurately characterize performance and compare the systems.

Types of Means There are three common formulas used for calculating a mean in performance analysis, and choosing the right one depends on what specific metric you are evaluating. Given a set of n real numbers $(x_1, x_2, \ldots, x_n)$, the three means and their use cases are as follows:

1. Arithmetic Mean (AM)

- **Formula:** $AM = \frac{1}{n}\sum_{i=1}^{n} x_i$
- **Use Case:** The AM is a good candidate for comparing the execution time performance of several systems. It is appropriate to use when the sum of all measurements is a meaningful and interesting value.
- **Key Difference:** When used for a time-based variable (like seconds), it has the important property of being directly proportional to the total time; if the total execution time doubles, the AM doubles. In your example, the AM for PC A is 20s, and for PC B is 13.33s, correctly showing that PC B took less total time on average.

2. Harmonic Mean (HM)

- **Formula:** $HM = \frac{n}{\sum_{i=1}^{n}\left(\frac{1}{x_i}\right)}$
- **Use Case:** The HM is the preferred method when you are calculating a mean value of *execution rates*, such as MIPS (Millions of Instructions Per Second) or MFLOPS (Millions of Floating-Point Operations Per Second).
- **Key Difference:** While the Arithmetic Mean is directly proportional to total time, the Harmonic Mean is inversely proportional to the total execution time, which is the exact mathematical property desired when dealing with rates.

3. Geometric Mean (GM)

- **Formula:** $GM = \sqrt[n]{x_1 \times \cdots \times x_n} = \left(\prod_{i=1}^{n} x_i\right)^{1/n}$
- **Use Case:** The GM is used to measure the *relative performance* of machines, specifically when dealing with normalized results or ratios (comparing a system under test to a baseline reference machine). It is the mean officially used by the SPEC benchmark consortium.

- **Key Difference:** GM gives consistent results regardless of which system is used as the reference baseline. Unlike the Arithmetic Mean, which can be heavily skewed by a few massive outliers, the Geometric Mean gives equal weight to all values in the dataset with respect to changes, making it far less biased by extreme outlier values.